

- Secrets
- ABAP
- Ansible
- Apex
- AzureResourceManager
- C
- C#
- C++
- CloudFormation
- COBOL
- CSS
- Dart**
- Docker
- Flex
- Go
- HTML
- Java
- JavaScript
- JCL
- Kotlin
- Kubernetes
- Objective C
- PHP
- PL/I
- PL/SQL
- Python
- RPG
- Ruby
- Scala
- Swift
- Terraform
- Text
- TypeScript
- T-SQL
- VB.NET
- VB6
- XML



Dart static code analysis

Unique rules to write Clean DART Code

All rules **115** Bug **15** Code Smell **100**

Tags ▾

Impact ▾

Clean code attribute ▾

Search by name...

Wildcard variable shouldn't be used	
Relative lib imports should not be used	
Uninitialized variables and fields should be explicitly typed	
Types should not be used as parameter names	
Null checks in equality operators should be avoided	
Unnecessary character escapes should be removed	
Redundant type casts should be removed	
Empty constructor bodies should be replaced with a semicolon	
"switch" statements should cover all cases	
Ternary operators should not be nested	
Iterable "whereType" should be used to filter by type	
Fields that are only assigned in the constructor should be "readonly"	

Wildcard variable shouldn't be used

Analyze your code

Consistency - Conventional Maintainability ⬇

Code Smell Major ⓘ

- Why is this an issue?
- How can I fix it?
- More Info

In Dart, you can use wildcard names(names consisting on underscores only: `_`, `__`, `___`, ...). Usually it should be used to name the variable/parameter that won't be directly used but needs to be declared. For example, as counter in the for-loop:

```
for (var _ in [1, 2, 3]) doSomething();
```

or as a field, in the pattern:

```
void sayHello(Person p) {
  switch (p) {
    case (name, _):
      print('Hello, $name!');
  }
}
```

Currently, this is possible to read such wildcard variables, for example, `print(_)`; . However, such behavior is supposed to change in the next versions of the Dart language, making such variables non-binding and breaking the code, where such variables are used. To avoid problems during upgrades, it's recommended to refactor the code, to not use wildcard variables.

What is the potential impact?

Once the breaking change is introduced, you will need to update all the code using wildcard variables or parameters to be able to upgrade to the latest Dart language versions.

Available In:

sonarcloud

